

UNIVERSIDAD

Carrera de Ingeniería Informática

PROYECTO DE ASIGNATURA

TEL420 — SISTEMAS PARALELOS

Simulador de Tráfico Vehicular Paralelo y Distribuido

Caso de aplicación: zona Plaza Principal – Mercado Central – Terminal de Buses, Yacuiba, Bolivia

Documento de diseño y planificación (borrador de trabajo)

Yacuiba, Bolivia

2026

1. Introducción y contexto

El presente documento describe el diseño y la planificación del proyecto de la asignatura TEL420 – Sistemas Paralelos, desarrollado en el marco de la actualización temática de la materia hacia un enfoque de Formación Basada en Competencias (FBC). El proyecto integrador consiste en un simulador de tráfico vehicular que aplica paradigmas de programación paralela y distribuida (memoria compartida, aceleración por GPU) sobre un caso de estudio real: una zona vial de la ciudad de Yacuiba, Bolivia.

El objetivo del proyecto no es únicamente construir una simulación funcional, sino demostrar de forma medible los tres criterios centrales del curso: eficiencia, escalabilidad y aplicación al contexto tecnológico local, tal como establece la competencia general de la asignatura.

1.1. Justificación de la temática elegida

De entre las opciones evaluadas (detector de objetos por GPU, sistema de mensajería distribuido, resolutor de problemas NP en paralelo y simulador de tráfico vehicular), se seleccionó el simulador de tráfico por ser la propuesta que mejor equilibra los tres criterios de evaluación: presenta una eficiencia claramente medible al comparar tiempos de ejecución secuenciales frente a paralelos, ofrece una escalabilidad muy visible al incrementar el número de vehículos simulados, y tiene una aplicación local directa al modelar una zona real y reconocible de la ciudad.

2. Definición del proyecto

2.1. Modelo de simulación

Se adopta un modelo de simulación de tráfico microscópico basado en agentes. Bajo este enfoque, cada vehículo es tratado como un agente independiente con posición, velocidad y ruta propias, cuyo estado se recalcula en cada paso de tiempo (tick) en función de reglas de comportamiento (velocidad máxima, distancia de seguridad respecto al vehículo precedente y respeto a intersecciones reguladas). Este modelo fue preferido sobre el enfoque macroscópico (que trata el tráfico como un flujo continuo mediante ecuaciones diferenciales) por su mayor claridad conceptual, su naturaleza intrínsecamente paralelizable —dado que el cálculo de cada agente es independiente— y su facilidad de comunicación ante el docente y los compañeros de curso.

2.2. Entorno de simulación: datos reales desde el inicio

A diferencia de un enfoque incremental que retrasa el uso de datos reales hasta etapas avanzadas, se optó por construir la simulación sobre el grafo real de calles de la zona seleccionada desde la primera etapa del proyecto, utilizando datos abiertos de OpenStreetMap. La complejidad visual (renderizado gráfico) se incrementa progresivamente en etapas posteriores, pero el entorno geográfico —calles, intersecciones y distancias— es real desde el primer sprint. Esta decisión reduce el riesgo de tener que adaptar la lógica de agentes a datos reales tardíamente en el desarrollo.

2.3. Zona de estudio: Yacuiba, Bolivia

Se delimita como área de estudio el sector comprendido entre tres puntos de referencia real de la ciudad de Yacuiba, elegidos por concentrar tipos de tráfico distintos y complementarios:

- Plaza Principal “12 de Agosto” — núcleo cívico con cruces e intersecciones de alta densidad.
- Mercado Central — zona comercial con flujo vehicular y peatonal mixto.
- Terminal de Buses (Av. San Martín) — nodo de tráfico pesado e intermitente por el ingreso y salida de buses interdepartamentales e internacionales.

Esta delimitación permite un modelo acotado, manejable en el tiempo de un semestre académico, sin perder representatividad de los patrones reales de congestión de la ciudad.

2.4. Tecnologías seleccionadas

Conforme a los módulos de la asignatura, se define el siguiente stack tecnológico, priorizando las herramientas estrictamente necesarias para demostrar los tres criterios de evaluación. Las tecnologías marcadas como opcionales se incorporarán únicamente si el cronograma lo permite.

Componente	Tecnología	Módulo del curso
Lógica de agentes y motor de simulación	Python	Base del proyecto
Datos reales de calles	OSMnx + NetworkX	Apoyo (obtención de datos)
Paralelismo en CPU (memoria compartida)	Multiprocessing / Numba / OpenMP	Módulo 2
Aceleración masiva por GPU	CUDA (PyCUDA o Numba CUDA)	Módulo 4
Visualización	Pygame o HTML/Canvas	Apoyo (interfaz)
Métricas y gráficas de rendimiento	Python (matplotlib)	Módulo 1
Simulación distribuida por sectores (opcional)	MPI (mpi4py)	Módulo 3
Empaquetado y despliegue (opcional)	Docker	Módulo 5

3. Requisitos del sistema

A continuación se especifican los requisitos del sistema, diferenciando explícitamente entre requisitos funcionales (qué acciones ejecuta el sistema), requisitos no funcionales (bajo qué estándares de calidad y comportamiento debe ejecutarlas) y elementos declarados fuera de alcance (qué queda deliberadamente excluido del proyecto).

3.1. Requisitos funcionales (RF)

Un requisito funcional describe una acción concreta y verificable que el sistema debe ejecutar.

ID	Descripción
RF01	El sistema debe cargar el grafo real de calles de la zona Plaza – Mercado – Terminal (Yacuiba) a partir de datos de OpenStreetMap.
RF02	El sistema debe generar N vehículos (agentes) con posición inicial, velocidad y ruta asignada dentro del grafo.
RF03	Cada vehículo debe recalcular su posición en cada paso de tiempo (tick), respetando velocidad máxima y distancia de seguridad respecto al vehículo precedente.
RF04	El sistema debe simular semáforos o reglas de prioridad en las intersecciones principales de la zona seleccionada.
RF05	El sistema debe ejecutar la simulación en modo secuencial (control), modo paralelo en CPU y modo GPU (CUDA), de forma seleccionable.
RF06	El sistema debe permitir iniciar la simulación.
RF07	El sistema debe permitir pausar y reanudar la simulación en cualquier momento.
RF08	El sistema debe permitir reiniciar la simulación, volviendo al estado inicial con los mismos parámetros.
RF09	El sistema debe permitir detener la simulación y volver a la pantalla de configuración.
RF10	El sistema debe permitir ajustar la velocidad de reproducción de la simulación (por ejemplo, 1x, 2x, 4x).
RF11	El sistema NO debe incluir una función de retroceso temporal (rebobinado) de la simulación; queda declarado fuera de alcance.
RF12	El sistema debe permitir configurar, antes de iniciar, el número de vehículos, el modo de ejecución y la duración de la simulación.
RF13	El sistema debe validar los parámetros ingresados por el usuario (por ejemplo, rechazar valores nulos o negativos).
RF14	El sistema debe mostrar, al finalizar o durante la ejecución, las métricas de tiempo de ejecución, velocidad promedio y nivel de congestión.
RF15	El sistema debe generar y almacenar gráficas comparativas de speedup y eficiencia entre los modos de ejecución evaluados.

ID	Descripción
RF16	El sistema debe permitir exportar los resultados de una corrida (imagen o archivo CSV) para su uso en el informe final.

3.2. Requisitos no funcionales (RNF)

Un requisito no funcional no describe una acción nueva, sino una cualidad, restricción o estándar de comportamiento que condiciona cómo se ejecutan los requisitos funcionales. Responden a la pregunta «con qué nivel de calidad, rendimiento o estabilidad debe funcionar el sistema», y no a «qué hace el sistema».

ID	Categoría	Descripción
RNF01	Rendimiento	La versión paralela (CPU) y la versión GPU deben mostrar una mejora de tiempo medible y estadísticamente relevante frente a la versión secuencial.
RNF02	Escalabilidad	El sistema debe soportar el incremento progresivo del número de vehículos (por ejemplo, de 100 a 10 000 o más) sin fallar, registrando métricas en cada nivel de carga.
RNF03	Portabilidad	El sistema debe poder ejecutarse en al menos dos entornos distintos (equipo local del desarrollador y un entorno con GPU disponible en la nube, por ejemplo Google Colab, en caso de no contar con GPU dedicada).
RNF04	Usabilidad	Los controles de la simulación (inicio, pausa, reinicio) deben responder en menos de un segundo, sin bloquear la interfaz.
RNF05	Mantenibilidad	El código debe estar modularizado en componentes independientes (agentes, motor de simulación, paralelismo, visualización, interfaz) para facilitar pruebas y modificaciones.
RNF06	Documentación	Cada módulo de paralelismo (CPU/GPU) debe estar comentado explicando qué parte del cómputo se distribuye y bajo qué criterio.
RNF07	Reproducibilidad	Las pruebas de rendimiento deben poder repetirse de forma consistente mediante el uso de semillas aleatorias fijas.
RNF08	Consistencia visual	La interfaz debe mantener el mismo estilo y disposición de controles en las distintas pantallas del sistema.

3.3. Fuera de alcance

Se declaran explícitamente fuera del alcance del proyecto los siguientes puntos, con el fin de delimitar expectativas y evitar ambigüedad respecto a lo que el sistema no está diseñado para hacer:

- No se implementará navegación GPS ni recálculo dinámico de rutas óptimas en tiempo real (tipo aplicaciones comerciales de navegación).

- No se simulará la totalidad de la ciudad de Yacuiba, únicamente la zona delimitada (Plaza – Mercado – Terminal).
- No se integrarán fuentes de datos de tráfico en vivo (cámaras, sensores IoT); todos los parámetros de la simulación son definidos y controlados por el desarrollador.
- No se representará el modelo de vehículos con nivel de detalle gráfico realista, ni se incluirán elementos identificativos como matrículas, marcas o modelos específicos; cada vehículo se representa como una entidad abstracta (agente).
- No se incluirá función de retroceso temporal (rebobinado) de la simulación.
- No se desarrollará una aplicación móvil ni una interfaz web pública; la visualización está pensada para ejecución local con fines de demostración académica.
- No se contempla una versión multiusuario ni concurrente; el sistema está diseñado para ser operado por un único usuario a la vez.
- No se generalizará el modelo a otras ciudades o zonas fuera del área delimitada, salvo disponibilidad de tiempo adicional.

4. Interfaz del sistema

El sistema contempla un mínimo de tres pantallas, diseñadas para cubrir el ciclo completo de uso: configuración de parámetros, ejecución interactiva de la simulación y visualización de resultados.

4.1. Pantalla 1 — Configuración

Permite definir, antes de iniciar la simulación: el número de vehículos, el modo de ejecución (secuencial, CPU paralelo o GPU) y la duración de la simulación en ticks. Corresponde a los requisitos RF12 y RF13.

4.2. Pantalla 2 — Simulación en vivo

Muestra la representación gráfica del grafo de calles con los vehículos en movimiento, junto con controles de reproducción: atrás (regresa a la pantalla de configuración, sin rebobinar la simulación), pausar/reanudar, reiniciar, y un control deslizante de velocidad de reproducción. Corresponde a los requisitos RF06 a RF10.

4.3. Pantalla 3 — Resultados

Presenta las métricas obtenidas de la corrida (tiempo de ejecución, speedup, velocidad promedio, nivel de congestión) mediante tarjetas de resumen, además de un botón para exportar los resultados en formato de imagen o CSV. Corresponde a los requisitos RF14 a RF16.

5. Historias de usuario

Las historias de usuario se redactan desde la perspectiva del desarrollador, dado que el proyecto corresponde a un trabajo individual de carácter académico. Cada historia se vincula a su requisito funcional o no funcional de origen, permitiendo trazabilidad completa entre la especificación y la planificación de sprints.

5.1. Épica 1 — Datos del entorno real

ID	Historia de usuario	Requisito
HU01	Como desarrollador, quiero descargar el grafo de calles de la zona Plaza – Mercado – Terminal usando OSMnx, para contar con una base real sobre la cual simular.	RF01
HU02	Como desarrollador, quiero identificar y marcar las intersecciones principales con semáforo o prioridad, para poder aplicarles reglas de tráfico.	RF04

5.2. Épica 2 — Motor de simulación base (secuencial)

ID	Historia de usuario	Requisito
HU03	Como desarrollador, quiero definir la estructura de un vehículo (posición, velocidad, ruta), para poder instanciar múltiples agentes.	RF02
HU04	Como desarrollador, quiero implementar la lógica de movimiento y distancia de seguridad por cada tick, para lograr un comportamiento realista.	RF03
HU05	Como desarrollador, quiero generar N vehículos con rutas aleatorias dentro del grafo, para poblar la simulación.	RF02
HU06	Como desarrollador, quiero ejecutar la simulación en modo secuencial como versión de control, para tener una base de comparación de tiempos.	RF05

5.3. Épica 3 — Paralelismo en CPU

ID	Historia de usuario	Requisito
HU07	Como desarrollador, quiero paralelizar el cálculo de posición de cada vehículo mediante multiproceso u OpenMP, para reducir el tiempo de cómputo por tick.	RF05
HU08	Como desarrollador, quiero medir el tiempo de ejecución con distinta cantidad de hilos o procesos, para calcular el speedup y la eficiencia.	RF14, RNF01

5.4. Épica 4 — Aceleración GPU

ID	Historia de usuario	Requisito
HU09	Como desarrollador, quiero portar el cálculo de posiciones a un kernel CUDA, para procesar miles de vehículos de forma simultánea.	RF05
HU10	Como desarrollador, quiero comparar el rendimiento entre CPU paralelo y GPU con distintas cantidades de vehículos, para demostrar la escalabilidad del sistema.	RF05, RNF02

5.5. Épica 5 — Métricas y análisis

ID	Historia de usuario	Requisito
HU11	Como desarrollador, quiero registrar automáticamente tiempo de ejecución, velocidad promedio y congestión por tramo en cada corrida, para obtener datos consistentes.	RF14, RNF07
HU12	Como desarrollador, quiero generar gráficas comparativas entre los modos secuencial, paralelo y GPU, para presentar resultados claros en el informe.	RF15

5.6. Épica 6 — Visualización

ID	Historia de usuario	Requisito
HU13	Como desarrollador, quiero renderizar los vehículos como puntos en movimiento sobre el grafo real de calles, para observar visualmente el comportamiento del sistema.	RF06 a RF10
HU14	Como desarrollador, quiero mejorar la visualización incorporando colores por velocidad o congestión y el fondo del mapa, para lograr una demostración final más clara.	RNF08

5.7. Épica 7 — Configuración y usabilidad

ID	Historia de usuario	Requisito
HU15	Como desarrollador, quiero definir los parámetros de simulación mediante un archivo de configuración o argumentos de línea de comandos, para no modificar el código en cada prueba.	RF12, RF13
HU16	Como desarrollador, quiero fijar semillas aleatorias, para que las pruebas de rendimiento sean reproducibles.	RNF07

6. Planificación por sprints

La planificación se organiza en cinco sprints, ordenados según la dependencia técnica entre historias de usuario: primero se construye el entorno y la lógica base, luego se incorpora el paralelismo en CPU, después la aceleración por GPU, y finalmente la visualización y el cierre del proyecto.

Sprint 1 — Entorno real y modelo base (2 semanas)

- HU01 — Grafo real de calles de la zona
- HU02 — Intersecciones y reglas de prioridad
- HU03 — Estructura de datos del vehículo
- HU04 — Lógica de movimiento y distancia de seguridad
- HU05 — Generación de N vehículos con rutas
- HU06 — Simulación secuencial de control

Entregable: Simulación secuencial funcionando sobre el mapa real de la zona seleccionada, sin visualización gráfica (verificación por datos y consola).

Sprint 2 — Paralelismo en CPU y primeras métricas (2 semanas)

- HU07 — Paralelización con multiproceso u OpenMP
- HU08 — Medición de speedup y eficiencia
- HU11 — Registro automático de métricas
- HU15 — Parámetros configurables sin modificar código
- HU16 — Semillas aleatorias fijas para reproducibilidad

Entregable: Versión paralela en CPU funcionando, con primeras gráficas comparativas de tiempo secuencial frente a paralelo.

Sprint 3 — Aceleración GPU y escalabilidad (2 a 3 semanas)

- HU09 — Kernel CUDA para cálculo de posiciones
- HU10 — Comparación CPU paralelo frente a GPU a gran escala
- HU12 — Gráficas comparativas completas

Entregable: Simulación ejecutándose sobre GPU con miles de vehículos, y conjunto completo de gráficas de rendimiento que demuestran los tres criterios de evaluación.

Sprint 4 — Visualización sobre el mapa real (2 semanas)

- HU13 — Visualización básica sobre el grafo real
- HU14 — Mejoras visuales: colores por congestión, fondo de mapa

Entregable: Demostración visual funcionando sobre la zona real de Yacuiba, con representación clara del estado de la simulación.

Sprint 5 — Cierre, documentación y presentación (1 a 2 semanas)

- Pruebas finales y corrección de errores
- Consolidación de la documentación técnica (RNF06)

- Publicación del código fuente en GitHub
- Preparación de la demostración en vivo y del informe final

Entregable: Proyecto completo, documentado, publicado en el repositorio y listo para su presentación y defensa.

7. Conclusiones y próximos pasos

El presente documento constituye la base de diseño y planificación del proyecto. Con el modelo, los requisitos, la interfaz, las historias de usuario y los sprints ya definidos, el siguiente paso corresponde a la implementación técnica, comenzando por el Sprint 1: la obtención del grafo real de calles de la zona seleccionada y la construcción de la lógica base de agentes en modo secuencial.

El código fuente del proyecto será alojado y versionado en un repositorio de GitHub, lo que permitirá llevar un control de cambios ordenado a lo largo de los sprints y facilitará la eventual revisión por parte del docente.

Referencias

- Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65, 126–139.
<https://doi.org/10.1016/j.compenvurbsys.2017.05.004>
- Grama, A., Gupta, A., Karypis, G., & Kumar, V. (2003). *Introduction to parallel computing* (2.^a ed.). Addison-Wesley.
- Kirk, D. B., & Hwu, W. W. (2016). *Programming massively parallel processors: A hands-on approach* (3.^a ed.). Morgan Kaufmann.
- Nagel, K., & Schreckenberg, M. (1992). A cellular automaton model for freeway traffic. *Journal de Physique I*, 2(12), 2221–2229. <https://doi.org/10.1051/jp1:1992277>
- OpenStreetMap contributors. (2026). OpenStreetMap [Conjunto de datos].
<https://www.openstreetmap.org>